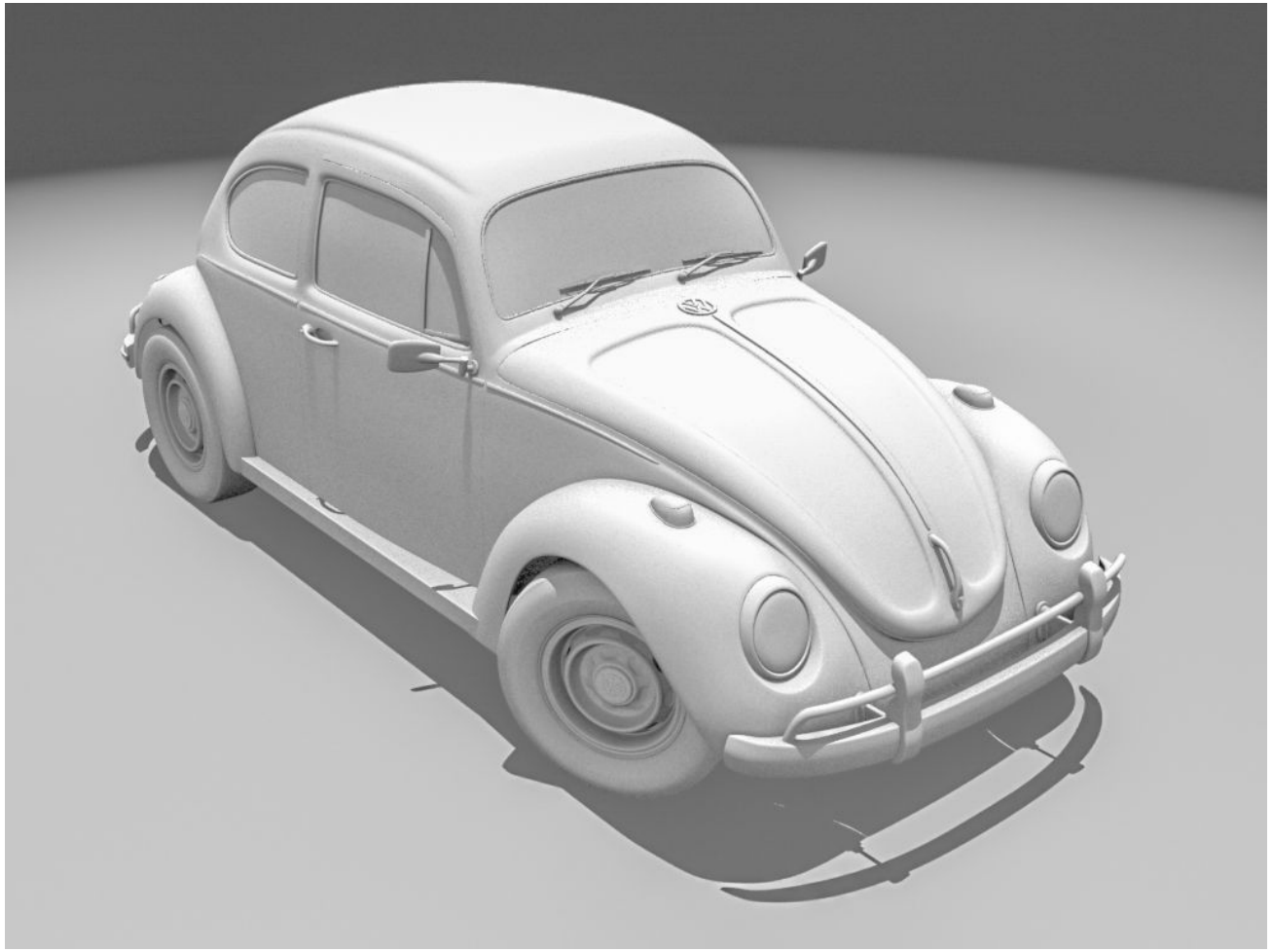
3D in Computer Graphics

- History
- Introduction
- Key concepts
- Application



History of 3D

Early Developments (1960s-1980s)

- The roots of 3D graphics can be traced back to the early days of computer graphics research in the 1960s.
- Ivan Sutherland's seminal work on Sketchpad in 1963 laid the foundation for interactive computer graphics, introducing concepts like graphical user interfaces and primitive 3D shapes.
- In the 1970s, the emergence of the first 3D wireframe models and rendering algorithms, such as hidden surface removal, marked significant progress in the field.

Introduction to 3D

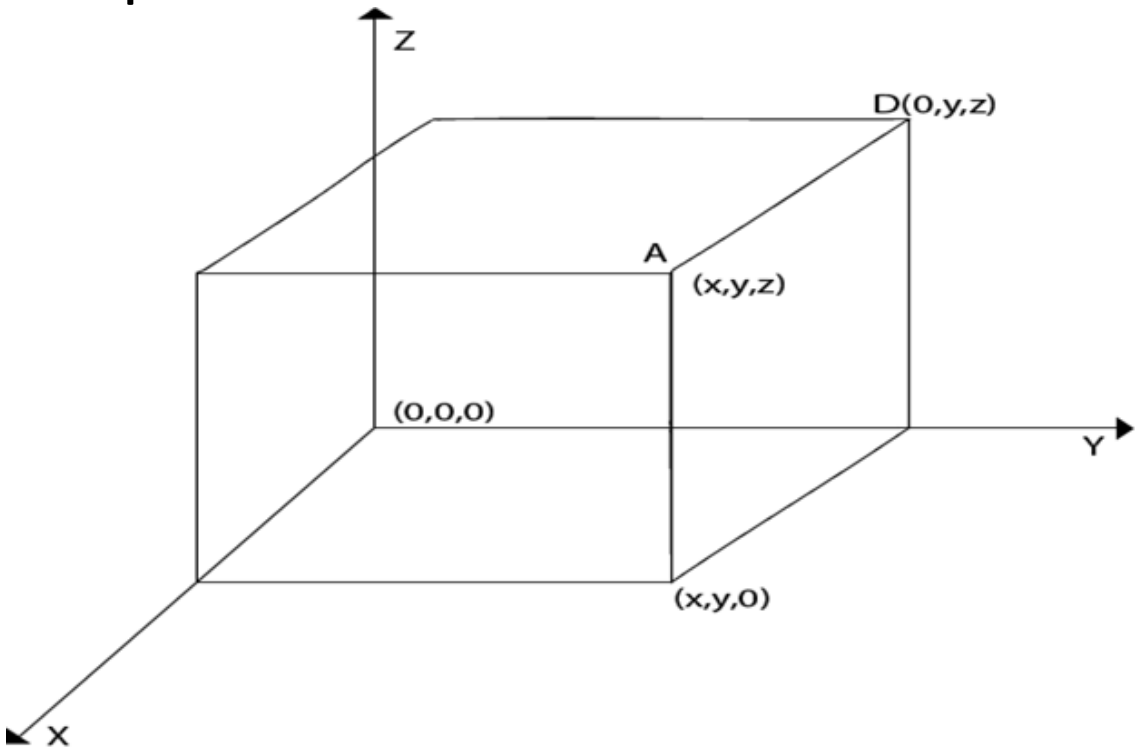
- The three-dimensional transformations are extensions of two-dimensional transformation.
- In 2D two coordinates are used, i.e., x and y whereas in 3D three co-ordinates x, y, and z are used.For three dimensional images and objects, three-dimensional transformations are needed. These are translations, scaling, and rotation.
- The 2D can show two-dimensional objects. Like the Bar chart, pie chart, graphs. But some more natural objects can be represented using 3D. Using 3D, we can see different shapes of the object in different sections.
- In 3D when a translation is done we need three factors for rotation also, it is a component of three rotations. Each can be performed along any three Cartesian axis.

Core Concepts of 3D Graphics

- **3D Models:** Digital representations of objects or scenes built from points, lines, and polygons.
- **Vertices, Edges, and Polygons:** The building blocks of 3D models. Vertices are like points in space, edges connect these points, and polygons (usually triangles) form the faces of the model.
- **Textures:** Digital images applied to the surfaces of 3D models to add details like color, patterns, and textures.
- **Lighting and Shading:** Techniques used to simulate how light interacts with the model's surfaces, creating depth, shadows, and a sense of realism.

3D Geometry

- The 3D coordinate system includes three axes: x, y, and z.
- The orientation can be right-handed or left-handed, affecting how the coordinates are interpreted and visualized.



Producing realism in 3D

- The three-dimensional objects are made using computer graphics.
- The technique used for two Dimensional displays of three Dimensional objects is called projection.
- Creating realistic 3D scenes is challenging but necessary for applications like CAD (Computer-Aided Design), which is used for manipulating machine components, automobile bodies, and aircraft parts.
- Realism in 3D graphics requires an understanding of depth and perspective.

3D Projections

- Parallel Projection
- Orthographic Projection
- Oblique Projection
- Isometric Projections
- Perspective Projection

Parallel projection

Characteristics:

- **Lines of Projection:** Parallel to each other and perpendicular to the projection plane.
- **Distortion:** No perspective distortion; objects maintain their relative proportions regardless of distance.

Applications:

- Used in Technical drawings, engineering designs, architectural blueprints.

Orthographic projection

Characteristics:

- **Lines of Projection:** Perpendicular to the projection plane.
- **Distortion:** Objects are shown in true scale without perspective distortion.
- **Views:** Typically includes top, front, and side views.

Applications:

- Used in Engineering and architectural drawings, CAD applications.

Oblique projection

Characteristics:

- **Lines of Projection:** Parallel but at an angle (not perpendicular) to the projection plane.
- **Distortion:** Offers a more pictorial effect than orthographic projection; partial depth effect.
- **Scale:** Maintains true scale along one axis (commonly the front face).

Applications:

- Used in Simple technical illustrations, early computer graphics, and games.

Isometric projection

Characteristics:

- **Lines of Projection:** Parallel and form equal angles (120 degrees) with the projection plane.
- **Distortion:** No distortion along the axes, maintaining uniform scale.
- **Views:** Provides a clear view of all three dimensions.

Applications:

- **Uses:** Video games (especially older or simpler games), technical and engineering drawings.

Perspective Projection

Characteristics:

- **Lines of Projection:** Converge at one or more vanishing points.
- **Distortion:** Mimics human eye perception; objects appear smaller as they get further away.
- **Depth:** Offers a realistic representation of depth and space.

Applications:

- Used in Realistic 3D rendering, video games, virtual reality, simulations, architectural visualization.

Rendering in 3D

- Rendering in computer graphics refers to the process of generating an image from a 3D model using specialized software or hardware.
- It involves the transformation of geometric data, textures, lighting, and other scene properties into a two-dimensional image that can be displayed on a screen or printed

Evolution of Rendering

- **Wireframe Rendering (1960s)**:
  - The earliest computer graphics systems rendered images using wireframe models consisting of simple geometric primitives such as points, lines, and polygons.
  - These wireframe models provided a basic representation of 3D objects but lacked realistic lighting and surface shading.
- **Scanline Rendering (1970s)**:
  - Scanline rendering algorithms were developed to rasterize and render polygonal models onto a 2D display.
  - This approach involved scanning each horizontal line of the screen and determining which polygons intersected with the scanline, applying shading and visibility calculations accordingly.
- **Hidden Surface Removal (1970s)**:
  - Techniques for hidden surface removal were introduced to address the problem of rendering occluded or obscured objects in a scene.
  - Algorithms such as the painter's algorithm and depth-buffering (z-buffering) were developed to determine which surfaces are visible from the viewer's perspective and render them accordingly.

Evolution of Rendering

- **Phong Shading (1973)**:
  - Phong shading, introduced by Bui Tuong Phong, revolutionized rendering by simulating specular highlights and smooth shading on 3D surfaces.
  - This shading model improved the visual realism of rendered images by accurately modeling the interaction between light sources and surface materials.
- **Ray Tracing (1980s)**:
  - Ray tracing emerged as a powerful rendering technique for generating photorealistic images by simulating the behavior of light rays in a scene.
  - Developed by researchers such as Turner Whitted and James Kajiya, ray tracing algorithms trace rays from the camera through each pixel, calculating intersections with objects and simulating lighting effects such as reflections, refractions, and shadows.
- **Global Illumination (1980s-1990s)**:
  - Global illumination algorithms were developed to simulate realistic lighting effects such as indirect illumination, color bleeding, and soft shadows.
  - Techniques such as radiosity and photon mapping were introduced to accurately model the complex interactions of light within a scene, enhancing the realism of rendered images.

Evolution of Rendering

- **Texture Mapping (1980s-1990s)**:
  - Texture mapping techniques were developed to apply 2D images (textures) onto 3D surfaces, adding detail and complexity to rendered objects.
  - Texture mapping enabled the simulation of surface materials such as wood, metal, and fabric, enhancing the visual richness of rendered scenes.
- **Real-Time Rendering (1990s-Present)**:
  - Advances in computer hardware and graphics processing led to the development of real-time rendering techniques capable of generating interactive graphics at high frame rates.
  - Real-time rendering is used in video games, simulations, virtual reality, and augmented reality applications, providing immersive experiences with dynamic lighting, shaders, and effects.
- **Physically Based Rendering (2000s-Present)**:
  - Physically based rendering (PBR) approaches aim to simulate light transport in a physically accurate manner, taking into account properties such as energy conservation, reflectance, and refraction.
  - PBR techniques are widely used in film production, architectural visualization, and product design, producing highly realistic images with accurate lighting and material properties.

Characteristics of Rendering

- **Realism**: Rendering aims to create images that closely resemble real-world scenes by accurately simulating lighting, shadows, reflections, refractions, and surface materials.
- **Efficiency**: Rendering algorithms strive to achieve high performance and scalability to handle complex scenes efficiently, allowing for real-time or interactive rendering in applications such as video games and virtual reality.
- **Flexibility**: Rendering systems often provide a variety of options and parameters to control the appearance of rendered images, including lighting models, material properties, anti-aliasing methods, and post-processing effects.
- **Accuracy**: Rendering algorithms should produce visually plausible results while minimizing artifacts such as aliasing, noise, and banding.
- **Interactivity**: In interactive applications, such as games and simulations, rendering algorithms need to provide responsive feedback to user input, often requiring trade-offs between visual quality and rendering speed.

Rendering Algorithms

- **Rasterization**: Rasterization is a fundamental rendering algorithm used to convert geometric primitives (points, lines, polygons) into pixels on the screen. It involves determining which pixels are covered by each primitive and interpolating attributes (such as color and depth) across the primitive's surface.
- **Ray Tracing**: Ray tracing is a rendering technique that simulates the behavior of light rays as they interact with objects in a scene. It traces rays from the camera through each pixel on the screen, calculating intersections with objects and computing lighting effects such as reflections, refractions, and shadows.

Rendering Algorithms

- **Shading Models**: Shading algorithms determine the color and brightness of pixels based on lighting conditions and surface properties.
- **Common shading models include:**
  - Lambertian Reflection: Simulates diffuse (matte) surfaces that reflect light equally in all directions.
  - Phong Reflection: Models specular (shiny) highlights by calculating the reflection direction based on surface normals and the viewer's direction.
  - Blinn-Phong Reflection: A variation of the Phong model that computes specular highlights more efficiently.

Rendering Algorithms

- **Global Illumination**: Global illumination algorithms simulate indirect lighting effects such as color bleeding, soft shadows, and ambient occlusion, which contribute to the realism of rendered images. **Examples include:**
  - **Radiosity:** Computes the transfer of light between surfaces by solving a system of equations representing the energy balance in the scene.
  - **Photon Mapping:** Uses a two-pass rendering approach to simulate indirect illumination by tracing photons emitted from light sources and then estimating the indirect lighting at each point in the scene.

Rendering Algorithms

- **Anti-Aliasing**: Anti-aliasing algorithms reduce the visual artifacts caused by aliasing, such as jagged edges (aliasing) and moiré patterns.
- Techniques include:
  - **Supersampling:** Renders the scene at a higher resolution and then downsamples the image to reduce aliasing.
  - **Multisampling:** Samples multiple points within each pixel and averages the results to produce smoother edges.
  - **Post-Processing:** Applies filters and blur algorithms to the rendered image to reduce aliasing and improve image quality.

Rendering Algorithms

- **Depth of Field**: Depth of field algorithms simulate the effect of a camera lens by blurring objects that are not in focus, creating a sense of depth and realism in the rendered image.
- **Motion Blur**: Motion blur algorithms simulate the blur effect caused by the movement of objects or the camera during exposure, enhancing the realism of dynamic scenes.

DirectX

- DirectX is a collection of application programming interfaces (APIs) developed by Microsoft for handling multimedia-related tasks, especially for gaming and multimedia content creation on the Windows platform.
- It provides developers with a set of tools and libraries to access hardware acceleration features of modern graphics cards and sound cards, enabling efficient rendering of 2D and 3D graphics, audio playback, input handling, and more.

Components of DirectX :

- **DirectDraw**: This component of DirectX is primarily used for 2D graphics rendering. It provides functions for drawing, text, and other 2D elements directly to the screen, bypassing the Windows Graphics Device Interface (GDI) for improved performance in games and multimedia applications.
- **Direct3D**: Direct3D is the 3D graphics component of DirectX, allowing developers to create and manipulate 3D graphics in real-time. It provides an interface for rendering 3D models, applying textures, performing lighting calculations, and implementing various rendering techniques such as shaders and effects.
- **DirectSound**: DirectSound is used for audio playback and manipulation in multimedia applications. It provides access to hardware-accelerated audio features, such as 3D positional audio effects, environmental effects, and hardware mixing, for immersive audio experiences in games and multimedia content.

Components of DirectX :

- **DirectInput**: DirectInput is responsible for handling input devices such as keyboards, mice, joysticks, and game controllers. It provides an abstraction layer for accessing input devices, enabling developers to create custom input handling logic for games and interactive applications.
- **DirectPlay**: DirectPlay was a networking API for multiplayer gaming, allowing developers to create networked multiplayer games with features such as matchmaking, session management, and voice chat. However, it has been deprecated in favor of other networking technologies.
- **DirectCompute**: DirectCompute is a GPU computing API that allows developers to harness the power of modern graphics hardware for general-purpose computing tasks, such as physics simulations, image processing, and machine learning algorithms.

Applications of DirectX

1. Gaming
2. Multimedia Content Creation
3. Virtual Reality (VR) and Augmented Reality (AR)
4. Simulation and Training
5. Education and Edutainment
6. Architectural and Engineering Visualization
7. Digital Signage and Advertising
8. Medical Imaging and Visualization

Open Graphics Library

- OpenGL is a cross-platform API for rendering 2D and 3D vector graphics. It provides a set of functions for interacting with graphics hardware to create high-performance, interactive graphics applications
- Developed by Silicon Graphics Inc. (SGI) in 1992, OpenGL was initially designed for SGI's workstations. It later evolved into an open standard managed by the Khronos Group, an industry consortium. Over the years, OpenGL has undergone several revisions, with updates to support new hardware capabilities and improve performance.

Components of OpenGL

- **Rendering Pipeline**: OpenGL follows a pipeline architecture for rendering graphics.
- It consisting of stages such as:

1. Vertex processing
2. Primitive assembly
3. Rasterization
4. Fragment processing
5. Framebuffer operations.

- The rendering pipeline, also known as the graphics pipeline, is a sequence of steps that a computer graphics system uses to create a 2D image from a 3D scene.
- This process involves transforming 3D models, applying textures and lighting, and ultimately converting the 3D representation into a 2D image that can be displayed on a screen.
- Here’s a breakdown of the main stages of the rendering pipeline:

OpenGL rendering pipeline:

1. **Application Stage**:
   - **Object Creation**: 3D models, textures, and scenes are created and defined using software applications.
   - **Scene Management**: Objects are placed in the scene, and the camera and lights are set up.

OpenGL rendering pipeline:

2. Model Transformation: Objects' coordinates are transformed from their local (object) space to world space.

- **View Transformation**: The scene is transformed from world space to camera (view) space, where the camera is positioned at the origin.
- **Projection Transformation**: The 3D view is projected onto a 2D plane using perspective or orthographic projection, converting 3D coordinates to 2D screen coordinates.
- **Clipping**: Objects or parts of objects outside the view frustum (the visible region) are clipped or discarded.
- **Viewport Transformation**: The 2D coordinates are mapped to the actual screen coordinates (pixels).

OpenGL rendering pipeline:

3.Rasterization:

- **Rasterization**: The 2D vertices of triangles are converted into pixels or fragments. This involves determining which pixels on the screen correspond to each triangle.
- **Fragment Generation**: Fragments are generated for each pixel covered by a triangle. Fragments contain data such as depth, color, and texture coordinates.

OpenGL rendering pipeline:

4.Fragment Processing:

- **Fragment Shader**: The properties of each fragment are processed, applying textures, colors, and lighting. This stage can also involve complex calculations for effects like bump mapping and procedural textures.
- **Depth Testing and Blending**: Fragments are tested for depth to determine visibility (depth buffering). Blending operations combine the fragment color with the existing pixel color to handle transparency and other effects.

OpenGL rendering pipeline:

5.Framebuffer Operations:

- **Frame Buffer Update**: The final color of each pixel is written to the frame buffer, where the complete image is stored.
- **Display**: The frame buffer is displayed on the screen, resulting in the final rendered image.

OpenGL Components:

- **Cross-Platform**: OpenGL is platform-independent, meaning it can run on different operating systems like Windows, macOS, Linux, and mobile platforms like Android and iOS.
- **Hardware Acceleration**: OpenGL utilizes hardware acceleration for rendering, leveraging the capabilities of graphics processing units (GPUs) to improve performance and efficiency.
- **Texture Mapping**: OpenGL supports texture mapping for applying images or patterns to 3D surfaces, enabling realistic rendering of materials, lighting effects, and textures.

OpenGL Components:

- **Extensibility:** OpenGL provides extension mechanisms that allow developers to access advanced features beyond the core API, enabling support for new hardware capabilities and rendering techniques.
- **Shading Language:** OpenGL includes a shading language (GLSL) for programming shaders, allowing developers to create custom vertex and fragment shaders for advanced rendering effects.
- **Buffer Objects:** OpenGL provides buffer objects for efficient storage and management of vertex data, textures, and other resources, optimizing memory usage and data transfer between the CPU and GPU.

OpenGL Components:

- **Transformation and Projection Matrices:** OpenGL offers functions for defining transformation and projection matrices, allowing developers to manipulate and transform geometric objects in 3D space.
- **Lighting and Material Properties:** OpenGL provides features for simulating lighting effects and defining material properties such as color, shininess, and transparency, enhancing the visual realism of rendered scenes.
- **Framebuffer Objects:** OpenGL supports framebuffer objects (FBOs) for off-screen rendering, enabling advanced rendering techniques like render-to-texture and post-processing effects.

Applications of OpenGL

1. Video Games
2. Computer-Aided Design (CAD) Software
3. Animation Software
4. Virtual Reality (VR) and Augmented Reality (AR)
5. Scientific Visualization
6. Geographic Information Systems (GIS)
7. Digital Content Creation (DCC) Tools
8. Educational Software
9. Medical Imaging and Visualization
10. Simulation and Training